

Assignment 2: In-Memory Workload Engine

ABDUBEK ALINA | SE2516

1. Implementation and complexity `DynamicArray` and `MinHeap` use primitive `int` arrays with initial capacity 4 and double capacity when full. `MyLinkedList` is singly linked and stores primitive `int` values in nodes. It keeps both head and tail, so appending is constant time. No Java collection is used to implement the structures. Indices are zero-based. Invalid indices throw `IndexOutOfBoundsException` and empty heap operations throw `IllegalStateException`.

Let n be the current number of elements and i a valid index. Average cases below assume uniformly distributed indices, a random relative order of distinct heap values, and a search target with a constant probability of being absent. Exact duplicate distribution can change heap averages. Space is temporary extra space per call, excluding the stored structure. All three structures store $\Theta(n)$ space; the array retains its peak capacity after removals.

Structure / operation	Best	Average	Worst	Auxiliary space	Reason
Array <code>add(x)</code>	$\Theta(1)$	$\Theta(1)$ amortized	$\Theta(n)$	$\Theta(1)$, $\Theta(n)$ on growth	Most writes are constant; doubling copies n values.
Array <code>add(i,x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$, $\Theta(n)$ on growth	Shifts $n-i$ values; the best case appends with spare capacity.
Array <code>remove(i)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Shifts $n-i-1$ values; removing the last needs no shifts.
Array <code>get(i)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	One direct indexed read.
Array <code>contains(x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Stops at a match, otherwise checks every element.
List <code>add(x)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Tail permits direct append.
List <code>add(i,x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head and append are constant; interior insertion finds predecessor.
List <code>remove(i)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Finds predecessor; removing head is constant.
List <code>get(i)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Traverses i links from head.
List <code>contains(x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Checks nodes until match or null.
Heap <code>insert(x)</code>	$\Theta(1)$	$\Theta(1)$ expected amortized*	$\Theta(n)$	$\Theta(1)$, $\Theta(n)$ on growth	Expected bubble distance is constant for random ranks; growth copies n cells.
Heap <code>peekMin()</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Minimum is stored at index 0.
Heap <code>extractMin()</code>	$\Theta(1)$	$\Theta(\log n)^*$	$\Theta(\log n)$	$\Theta(1)$	Replacement may descend the heap height; equal values can stop immediately.

Heap insertion has $O(\log n)$ amortized worst-case cost, even though a particular resizing call costs $\Theta(n)$. Without resizing its worst case is $\Theta(\log n)$. Expected insertion cost relies on the random-rank model, and is not a guarantee for arbitrary inputs. Extraction average assumes random distinct priorities. Empty operations and rejected inputs terminate in $\Theta(1)$, with exceptions. The inspection helper `isValid()` is $\Theta(n)$ worst-case time and $\Theta(1)$ space; `size()`, `metrics()` and `reset()` are $\Theta(1)$.

Doubling gives amortized constant append: copies over capacities $4, 8, \dots, 2^k$ total less than 2 times the final capacity. This amortized bound is different from a single-operation average over indices.

2. Loop invariant proofs ### `DynamicArray.contains` Invariant: Before iteration i , every cell with index j in $[0, i)$ has been checked and `data[j]` is different from x . The logical array and size remain unchanged.

Initialization: $i=0$, so the checked prefix is empty and the statement is true.

Maintenance: The method reads `data[i]` and compares it with x . If they are equal it returns true, which is correct. Otherwise the checked prefix extends through i and the invariant holds for $i+1$.

Termination: i increases by one and is bounded by size. If no early return happens, $i=size$ and the invariant says every logical element is different from x , so false is correct.

Conclusion: A true result has an actual matching element; a false result means the whole logical array was checked. Therefore `contains` is correct.

`DynamicArray.remove` Let s be the original size and A a snapshot of the original logical array. After saving `A[index]`, the loop starts at $i=index$.

Invariant: Before each iteration i , `data[j]=A[j+1]` for $index \leq j < i$. Cells $j < index$ are unchanged, and `data[j]=A[j]` for $i \leq j < s$. The saved value is `A[index]`.

Initialization: $i=index$. The shifted interval is empty and all array cells still have their original values, so the invariant holds.

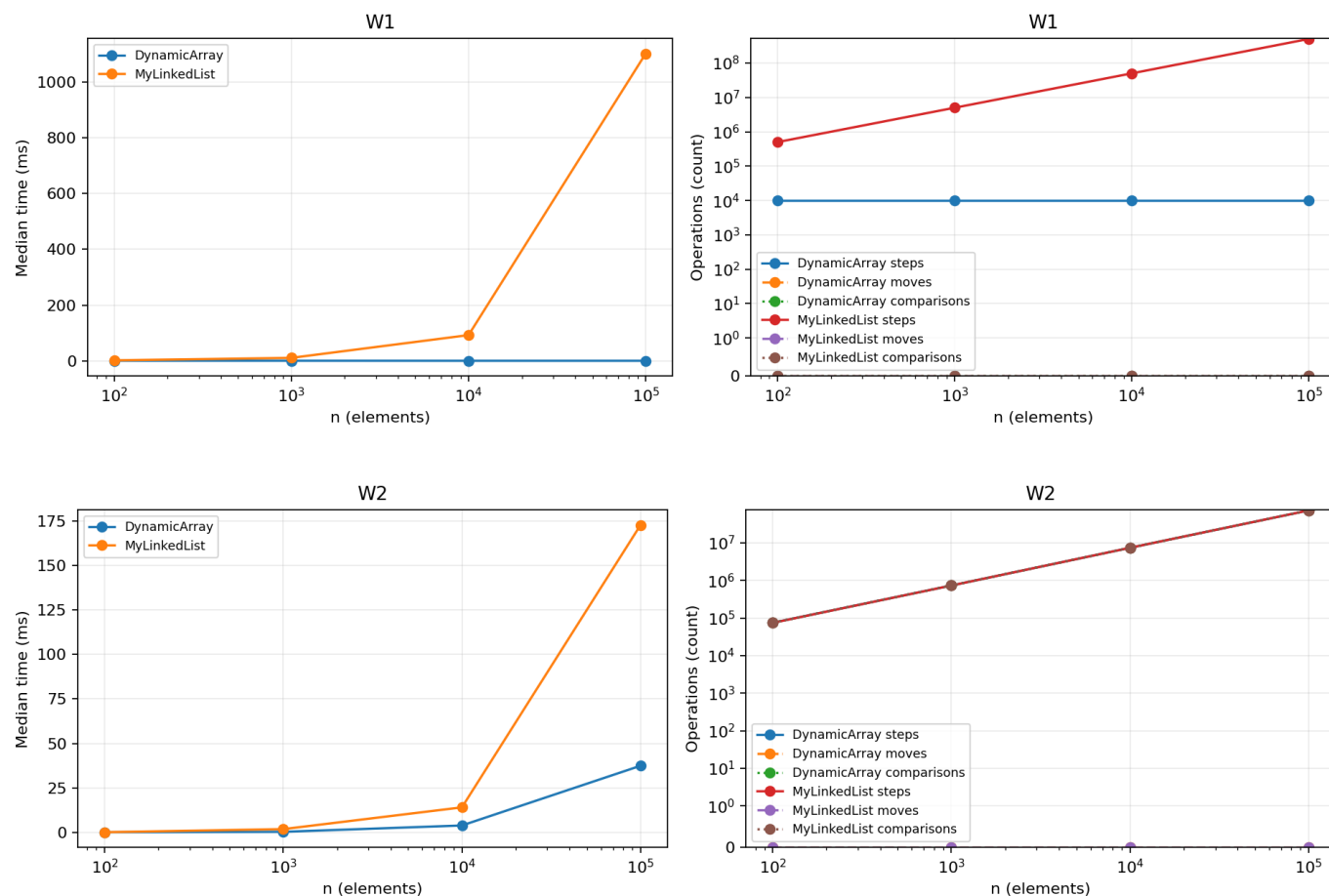
Maintenance: The loop assigns `data[i]=data[i+1]`. Since $i+1$ is in the unmodified suffix, it reads `A[i+1]`. This extends the shifted interval by one while leaving the remaining suffix unchanged. After i increases, the invariant holds again.

Termination: i increases and stops at $s-1$. Thus every destination from $index$ through $s-2$ contains the following original element. The method decreases size to $s-1$, making the stale last cell irrelevant.

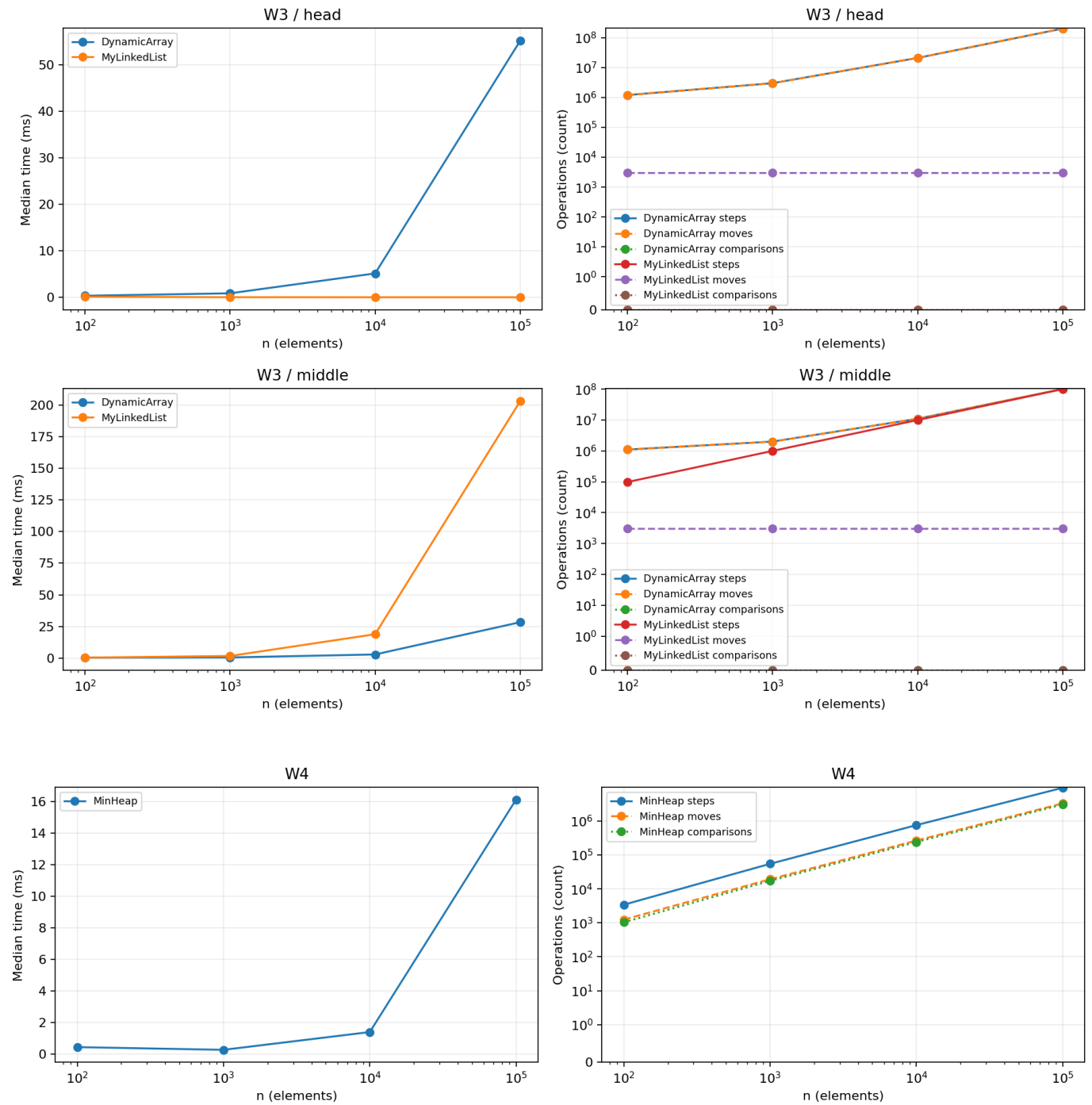
Conclusion: The returned value is the requested original element. All other elements keep their order and the logical size decreases exactly once, proving removal is correct.

3. Benchmark method and plots Sizes are 100, 1000, 10000 and 100000. At each size new Random(42) generates the same nonnegative input for every structure. W1 uses 10000 precomputed random indexes. W2 uses 1000 queries, exactly 500 sampled present values and 500 negative missing values. W3 inserts 1000 values and then removes 1000 values at fixed index 0 or original $n/2$. W4 inserts n values, extracts n values and checks nondecreasing order.

Each case discards two full warm-up runs, then records five runs on fresh structures. CSV time is the median; counters are taken from that same run. Preparation is excluded from W1-W3 but both heap construction and extraction are included in W4. Steps, moves and comparisons are counted inside methods. Array copying and shifting count a read and a move. List traversal counts a next step; reference changes count moves. Heap swaps count two reads and two moves. Validation comparisons outside structure methods are excluded from counters. Timing still includes counter increments and W4 order validation. Measurements are empirical results from this environment and can change on another computer.



3. Benchmark plots (continued)



4. Discussion DynamicArray get reads one cell, whereas a linked-list get follows i links. This makes random list access more expensive as n grows. Array elements are adjacent in memory, so one cache line can provide several nearby values. Sequential array search benefits from spatial locality and hardware prefetching. Linked nodes are separate objects and may be far apart in memory. Following a pointer depends on the preceding node load, which limits overlap between memory reads. Even equal search comparison counts therefore do not guarantee equal execution time. Node objects also have headers, references and alignment overhead. Creating many nodes creates allocation work and can increase garbage collection pressure. W3 head operations favour the list because reference updates are constant while arrays shift most values. Middle list operations still traverse many links, so changing only two references does not make the full operation constant time. The array can be competitive in the middle because contiguous shifts have better locality than pointer traversal. A linked list is useful for repeated head changes or appending with a stored tail. MinHeap is useful when jobs must be repeatedly removed in priority order without sorting after every insertion. Small timings can fluctuate because of JVM compilation, scheduling and garbage collection, so median measurements and operation counts should be considered together.

5. Validation and submission JUnit tests compare random list operations against ArrayList and heap operations against PriorityQueue. They cover empty structures, one element, duplicates, end positions, invalid indices, extreme integers, and exact counter examples. Heap property is checked after every operation in the randomized heap test, and draining verifies sorted output. The local repository contains main, feature/array, feature/list, feature/heap and feature/metrics, with release tag v1.0. A GitHub remote and its submission link still need to be created by the account owner. Bonus memory measurements and Floyd buildHeap are excluded.

Measured times at $n=100000$

Workload	Variant	Structure	Median ms
W1	-	DynamicArray	0.086839
W2	-	DynamicArray	37.488292
W3	head	DynamicArray	55.119789
W3	middle	DynamicArray	28.347542
W1	-	MyLinkedList	1102.887836
W2	-	MyLinkedList	172.625698
W3	head	MyLinkedList	0.014750
W3	middle	MyLinkedList	203.033778
W4	-	MinHeap	16.142077

Validation: all four JUnit 5 tests passed on Java 17; all 36 benchmark cases completed. Compilation used Eclipse ECJ and tests used JUnit Platform standalone because Maven was unavailable in this environment. Maven configuration is included for local builds.